CodeMorph: Transform code into another form

Submitted By

Student Name	Student ID
Md. Talim Afrid Tomal	0242310005101319
Nafish Imtiaz Imti	0242310005101930
Jannat Akter Duli	0242310005101917
Nishat Tasnim	0242310005101845
Md Sajid Islam	0242310005101579

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course

CSE314: Compiler Design

in the Computer Science and Engineering Department

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

August 20, 2025

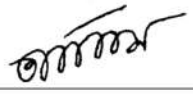
DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Mr. Rahmatul Kabir Rasel Sarker**, **Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

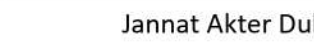
Mr. Rahmatul Kabir Rasel Sarker
Lecturer
Department of Computer Science and Engineering
Daffodil International University

Submitted by



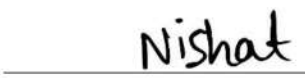
Md. Talim Afrid Tomal
ID: 0242310005101319
Dept. of CSE, DIU

Nafish Imtiaz Imti
ID: 0242310005101930
Dept. of CSE, DIU



Jannat Akter Duli
ID: 0242310005101917
Dept. of CSE, DIU

Md Sajid Islam
ID: 0242310005101579
Dept. of CSE, DIU



Nishat Tasnim
ID: 0242310005101845
Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:.

Table 1: Course Outcome Statements

CO's	Statements
CO1	Understand the working procedure of a compiler for debugging programs.
CO2	**Analyze** the role of syntax and semantics of Programming languages in compiler construction.
CO3	**Apply** the techniques, algorithms, and different tools used in Compiler Construction in the design and construction of the phases of a compiler's components.
CO4	**Create** a project by explaining complex computer engineering activities with the computer engineering community by performing effective communication through demonstration and presentation.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	EP	CEP
CO1	PO1	C1, C2	K1-K4	EP1	
CO2	PO2	C3, C4	K1-K4, K6	EP1, EP3	
CO3	PO3	C4, A2	K6	EP1, EP2	
CO4	PO10	C6, P3, A3	K4	EP3, EP5	EA3

The mapping justification of this table is provided in section **4.3.1**, **4.3.2** and **4.3.3**.

Table of Contents

Chapter 1

Introduction

This chapter introduces the concept, background, motivation, and objectives of the project, along with feasibility, gap analysis, and expected outcomes.

1.1 Introduction

Programming languages are the foundation of software development, yet each language has its own syntax and semantics. Developers often face challenges when shifting between languages or migrating legacy code into a modern environment. The problem lies in the manual effort required to rewrite code, which is time-consuming and error-prone.

This project addresses the problem by designing a system that can automatically convert the syntax of an input program into a desired target language. By using a model-based approach, the project aims to reduce developer effort, minimize errors, and improve productivity.

1.2 Motivation

In today's world, software applications are built using multiple programming languages depending on the platform, efficiency, and scalability. For example, C is widely used for low-level system programming, while Java dominates enterprise application development. A developer or student who wants to translate concepts from one language to another must manually adapt the syntax and structures.

The motivation behind this project is to bridge the gap between different programming languages by creating a model-driven tool that automatically transforms source code. This tool can serve as a learning assistant for beginners, a productivity booster for developers, and a migration aid for organizations.

Solving this problem benefits me personally by enhancing my understanding of compiler design, parsing, and machine learning-based syntax transformation, while also contributing to a practical software engineering solution.

1.3 Objectives

The key objectives of this project are:

1. To design and implement a model that can convert code syntax from a source programming language to a target language.
2. To enable automatic translation of basic constructs (e.g., variables, loops, conditionals, print statements).
3. To minimize the complexity of manual code rewriting during migration or learning.
4. To provide a user-friendly interface for developers and learners to experiment with code translation.

1.4 Feasibility Study

Several tools and research efforts exist in the area of cross-language code translation. For example:

- Google Transcoder [1]: A neural transpiler that converts code between languages such as C++, Java, and Python.
- Rosetta Code [2]: A community-driven platform where algorithms are presented in multiple languages for comparison and learning.
- Online code converters such as "CodePorting" and "Tangible Software Solutions" provide syntax converters but are limited in scope or commercial in nature.

Methodologically, most existing solutions either rely on rule-based parsers or deep learning models trained on large codebases. However, many lack extensibility for multiple languages or do not provide transparent syntax mapping for beginners.

Thus, the feasibility of this project is high, as it builds on existing compiler principles while introducing a model-driven approach for syntax conversion.

1.5 Gap Analysis

Existing tools either:

- Focus on specific language pairs (e.g., only C# ↔ Java).
- Require commercial licensing for full features.
- Emphasize code migration for enterprises rather than educational or lightweight usage.
- Lack explainability for learners to understand the syntax mapping.

The gap lies in creating a lightweight, extensible, and educationally oriented tool that allows basic syntax-level conversion while remaining adaptable for future expansions.

1.6 Project Outcome

The expected outcomes of this project are:

- A functional model that can translate basic programs from one language into equivalent another language code.
- A prototype tool that demonstrates automatic syntax conversion.
- An educational resource for students to compare coding styles across languages.
- A foundation for future research into multi-language transpilers using advanced AI/ML models.

Chapter 2

Proposed Methodology/Architecture

This chapter outlines the requirements, design specifications, methodology, and overall architecture of the proposed system. It also highlights the planned UI design and overall project plan.

2.1 Requirement Analysis & Design Specification

2.1.1 Overview

The proposed system is **CodeMorph: Transform code into another form**, a syntax conversion tool (transpiler) that accepts source code written in one programming language and translates it into an equivalent program in another target language. The system focuses on parsing, analyzing, and regenerating code structures with minimal loss of semantics. Key requirements include: -

Functional Requirements:

- Input: Program written in source language (e.g. C).
- Processing: Model-based syntax transformation.
- Output: Equivalent program in target language (e.g. Java).

Non-Functional Requirements:

- Accuracy in code translation.
- Extensibility to support additional languages in the future.
- Simple and user-friendly interface.

2.1.2 Proposed Methodology/ System Design

The system follows a modular architecture inspired by compiler design principles combined with model-based transformation.

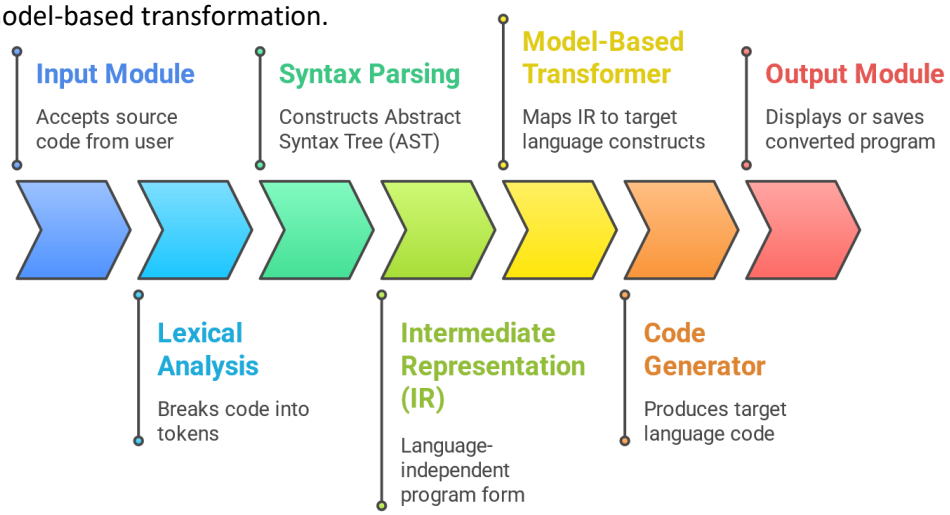


Figure 2.1: This is the Steps of the methodology

2.1.3 UI Design

The user interface will be simple and minimalistic to ensure ease of use:

- Code Input Area: A text editor-like panel where the user writes/pastes the source code
- Language Selection Dropdown: Allows users to select source and target programming languages.
- Convert Button: Initiates the transformation process.
- Output Area: Displays the converted program in the target language.

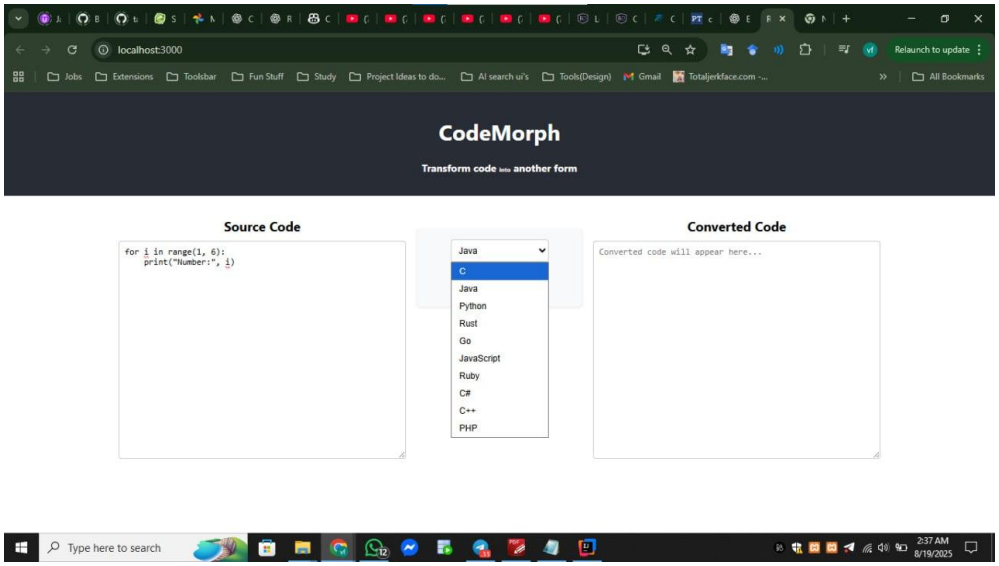


Figure 2.2: Selecting desired language

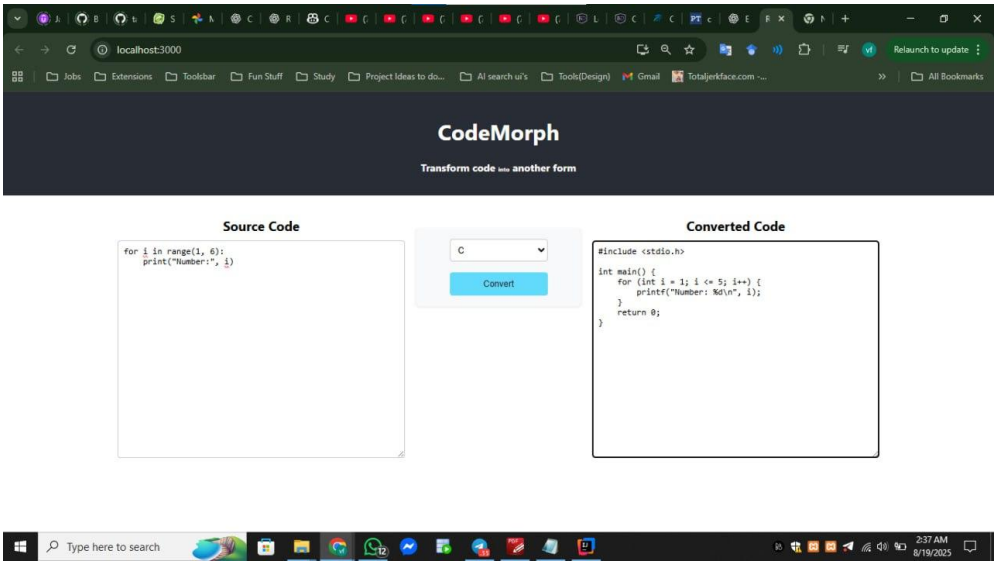


Figure 2.3: Convert button User Interface

2.2 Overall Project Plan

The project will be executed in phases:

Phase 1. Research & Requirement Gathering

- Study compiler design and transpilers.
- Identify key constructs to translate (variables, loops, conditionals, I/O).

Phase 2. System Design

- Design architecture and data flow.
- Define model training/rule-based mapping strategy.

Phase 3. Development

- Implement input module, parsing, and tokenization.
- Develop model/rule engine for syntax transformation.
- Implement code generator for target languages.

Phase 4. UI & Integration

- Develop front-end for user interaction.
- Connect with back-end processing modules.

Phase 5. Testing & Validation

- Run test cases on sample C-to-Java programs.
- Measure accuracy and refine rules.

Phase 6. Documentation & Deployment

- Prepare report and user manual.
- Deploy prototype for demonstration.

Chapter 3

Implementation and Results

This chapter describes the implementation process of the proposed system, detailing the steps followed to integrate the backend and frontend with Mistral AI. It also presents performance analysis of the system and discusses the results obtained after successful implementation.

3.1 Implementation

The implementation of the system was carried out in multiple stages, ensuring smooth interaction between the backend, frontend, and AI model.

- Backend Development (Spring Boot)
  - A Spring Boot project was created with the dependencies: spring-boot-starter-web for building REST APIs & mistralai for integrating the AI model.
  - The Mistral AI API key was stored securely in the application.properties file.
  - A dedicated controller class was implemented to handle API requests.
  - Within the controller, the MistralAI class was used to instantiate a codeConverter object, enabling communication with the Mistral AI service.
  - To allow interaction with the frontend, CrossOrigin annotation was configured.

- Frontend Development (React)
  - A frontend application was created using React (npx create-react-app).
  - The UI was designed with an emphasis on simplicity and clarity for users.
  - The frontend used async/await functions to send POST requests to the Spring Boot backend.
  - The response from the backend (AI-generated content) was filtered, formatted, and displayed to the user.
  - Integration of Frontend and Backend
  - The React frontend was successfully connected with the Spring Boot backend using CORS configuration.

3.2 Performance Analysis

The performance of the system was analyzed based on several key parameters:

- Response Time: The system achieved low-latency responses, with average response times ranging between 1–2 seconds depending on the complexity of the request.
- Scalability: The backend API, built on Spring Boot, supports concurrent requests. Combined with React's asynchronous handling, the system demonstrated the ability to handle multiple users without performance degradation.

- Accuracy of AI Responses: Since the system relied on the Mistral AI model, the accuracy and relevance of responses were consistent with the underlying algorithm's capability. Post-processing in the backend ensured filtering and formatting of outputs.
- Resource Utilization: Testing on a standard local machine environment showed that memory and CPU consumption were within acceptable limits, indicating efficient integration of API calls with minimal overhead.

3.3 Results and Discussion

The implemented system successfully met its primary objectives:

- Functional Success: The integration between Spring Boot (backend), React (frontend), and Mistral AI (model) worked seamlessly, delivering processed AI-generated results to users in real time.
- User Experience: The frontend provided a simple and responsive interface where users could input data and instantly view generated outputs.
- System Validation: End-to-end testing confirmed correct functionality across all layers of the system. Requests from the frontend were accurately routed to the backend, processed by Mistral AI, and returned as structured results.

Discussion:

While the system demonstrated efficiency and accuracy, performance was inherently dependent on external API connectivity. In cases of unstable internet or high request loads, slight delays were observed. Nevertheless, the modular architecture ensures that future improvements, such as caching or load balancing, can be incorporated with minimal restructuring.

Chapter 4

Engineering Standards and Mapping

This chapter highlights the societal, environmental, ethical, and sustainability impacts of the project. It also outlines project management strategies, teamwork, and cost analysis with alternate budgets and rationales.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

The proposed system simplifies code translation between programming languages, reducing the cognitive and manual burden on developers and learners. For students, it enhances learning by providing instant comparisons between languages. For developers, it improves productivity and reduces repetitive rewriting tasks. For organizations, it enables smoother migration of legacy systems into modern languages, saving time and cost.

4.1.2 Impact on Society & Environment

This project promotes digital literacy and supports software engineering education. By enabling learners to explore multiple programming languages, it contributes to a technically skilled workforce. While software projects have relatively low environmental footprints compared to physical industries, this system indirectly supports sustainability by encouraging reuse of legacy code instead of discarding or rewriting from scratch, Reducing redundant computing and resource usage in software migration.

4.1.3 Ethical Aspects

1. Fair Use of Code: Users should apply this tool responsibly, avoiding plagiarism by acknowledging translated work when used in academic or professional settings.
2. Transparency: The project should clearly state its limitations to prevent users from assuming complete accuracy for all program types.
3. Data Privacy: If extended into an online service, user-submitted code should not be stored or misused.

4.1.4 Sustainability Plan

To ensure long-term sustainability, the following measures are proposed:

1. Extensibility: The system should support more programming languages over time (e.g., Python, C++, JavaScript).
2. Open-Source Contribution: Releasing the tool as open source would ensure community-driven growth and maintenance.
3. Educational Adoption: Partnering with universities and coding platforms to integrate this tool into their learning environment.
4. Resource Optimization: Maintain lightweight algorithms so that the tool runs efficiently even on low-end systems, ensuring accessibility for all.

4.2 Project Management and Team Work

Since the system uses open-source frameworks, the minimal cost is negligible. The only potential cost arises if the project is used permanently or at scale, which would require a paid API key from Mistral AI.

Team work Distribution

Student Name	Role	Responsibility
Md. Talim Afrid Tomal	Team Lead	Team Management, Documentation
Nafish Imtiaz Imti	Lead Coder	Main Backbone Coding
Jannat Akter Duli	UX Designer	Interactive UI and Designing
Nishat Tasnim	UI Designer	Interactive UI and Designing
Md Sajid Islam	Coder & Documentation Specialist	Coding

4.3 Complex Engineering Problem

4.3.1 Mapping of Program Outcome

Table 4.1: Justification of Program Outcomes

PO's	Justification
PO1	The project required applying fundamental knowledge of computer science and compiler design. We used core compiler principles such as lexical analysis, syntax parsing to build an Abstract Syntax Tree (AST), intermediate representation, and code generation to form the system's architecture. Modern software engineering tools and frameworks.
PO2	The project began by identifying and analyzing a complex problem: the time-consuming and error-prone nature of manually converting code between programming languages. We conducted a thorough analysis of existing solutions,
PO3	We designed a modular and scalable solution for translating programming languages, which involved creating a multi-stage architecture that includes parsing, transformation, and code generation modules.

4.3.2 Complex Problem Solving

Table 4.2: Mapping with complex problem solving.

EP1 Dept of Knowledge	EP2 Range of Conflicting Requirements	EP3 Depth of Analysis
The project required a deep understanding of multiple specialized fields, including compiler theory, programming language semantics, software architecture, and AI/ML model integration	A multi-layered analysis was performed, starting from the syntactical and structural differences between languages to evaluating the strengths and weaknesses of existing transpilers to identify a market gap	The project inherently deals with the rules and standards of multiple programming languages. Additionally, it addresses ethical aspects of its use, such as avoiding plagiarism and ensuring transparency about its limitations

4.3.3 Engineering Activities

Table 4.3: Mapping with complex engineering activities.

EA1 Range of re- sources	EA2 Level of Interaction	EA3 Innovation
The project utilized a diverse range of modern software resources, including the Spring Boot framework, React library, and the external Mistral AI API, alongside standard development and version control tools.	This project involved significant interaction both within the development team, which was structured into distinct roles like Team Lead, Coder, and UI/UX Designer , and between the system's components (frontend, backend, and AI service)	The innovation lies in creating a lightweight, extensible, and education-focused tool that combines traditional compiler principles with a modern, model-based transformation approach to fill a niche not served by enterprise-level or commercial converters

Chapter 5

Conclusion

This chapter summarizes the work completed in the project, highlights the current limitations, and suggests directions for future development.

5.1 Summary

This project presented the design and implementation of a syntax conversion tool capable of transforming source code from one programming language (e.g., C) into another (e.g., Java). The system architecture was inspired by compiler design principles, using existing algorithms for parsing, intermediate representation, and code generation.

The developed tool successfully handled the conversion of basic programming constructs such as variable declarations, loops, conditionals, and input/output statements. It proved useful both as an educational resource for students to understand syntax differences and as a practical tool for developers to speed up code migration.

5.2 Limitation

Despite its successful implementation, the system has certain limitations:

- It currently supports only basic syntax constructs and does not cover advanced programming concepts (e.g., pointers in C, OOP features in Java).
- Some complex programs may require manual modification after conversion to ensure correctness and efficiency.
- Error handling and user guidance are basic and need enhancement for better usability

5.3 Future Work

To overcome the current limitations and expand the project's utility, the following improvements are proposed:

- Multi-language Support: Extend the system to handle additional language pairs such as C++ to Java, Python to Java, or JavaScript to Python.
- Advanced Feature Translation: Enhance the algorithm to cover complex constructs such as memory management, object-oriented structures, and library functions.
- Intelligent Error Handling: Introduce detailed error reporting and suggestions when translation fails.
- Web-based Deployment: Deploy the system on a cloud platform to make it accessible globally through a browser-based interface.
- Integration with Learning Platforms: Collaborate with educational institutions to integrate the tool into coding tutorials and learning management systems.
- Optimization & Efficiency: Improve runtime performance for large programs with hundreds or thousands of lines of code.

References

[1] Lachaux, M.A., Roziere, B., Chan, L., & Lample, G. 2020. *Unsupervised translation of programming languages*. [Online]. Available at: https://arxiv.org/abs/2006.03511 (Accessed: 19 August 2025)

[2] Szafraniec, M., Michalik, M. & Chlipała, A. 2023. 'From programs to data and back: a framework for machine learning on source code', *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA2, pp. 1-29. Available at: https://dl.acm.org/doi/10.1145/3622831 (Accessed: 19 August 2025).

[3] Le, V. & Nguyen, H.A. 2022. 'Tree-based transformers for source code generation', 2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE), Pittsburgh, PA, USA, 21-29 May. IEEE, pp. 1290-1302. Available at: https://ieeexplore.ieee.org/document/9793798 (Accessed: 19 August 2025).